4.5 Designing Recursive Algorithms

In general, an algorithm that uses recursion typically has the following form:

- ***Test for base cases.*** We begin by testing for a set of base cases (there should be at least one). These base cases should be defined so that every possible chain of recursive calls will eventually reach a base case, and the handling of each base case should not use recursion.
- ***Recur.*** If not a base case, we perform one or more recursive calls. This recursive step may involve a test that decides which of several possible recursive calls to make. We should define each possible recursive call so that it makes progress towards a base case.

Parameterizing a Recursion

To design a recursive algorithm for a given problem, it is useful to think of the different ways we might define subproblems that have the same general structure as the original problem. If one has difficulty finding the repetitive structure needed to design a recursive algorithm, it is sometimes useful to work out the problem on a few concrete examples to see how the subproblems should be defined.

A successful recursive design sometimes requires that we redefine the original problem to facilitate similar-looking subproblems. Often, this involved reparameterizing the signature of the function. For example, when performing a binary search in a sequence, a natural function signature for a caller would appear as binary_search(data, target). However, in Section 4.1.3, we defined our function with calling signature binary_search(data, target, low, high), using the additional parameters to demarcate sublists as the recursion proceeds. This change in parameterization is critical for binary search. If we had insisted on the cleaner signature, binary_search(data, target), the only way to invoke a search on half the list would have been to make a new list instance with only those elements to send as the first parameter. However, making a copy of half the list would already take $O(n)$ time, negating the whole benefit of the binary search algorithm.

If we wished to provide a cleaner public interface to an algorithm like binary search, without bothering a user with the extra parameters, a standard technique is to make one function for public use with the cleaner interface, such as binary_search(data, target), and then having its body invoke a nonpublic utility function having the desired recursive parameters.

You will see that we similarly reparameterized the recursion in several other examples of this chapter (e.g., reverse, linear_sum, binary_sum). We saw a different approach to redefining a recursion in our good_fibonacci implementation, by intentionally strengthening the expectation of what is returned (in that case, returning a pair of numbers rather than a single number).